addac112 LOOPS

This is the addac112 looper firmware with a different engine in the left knob section: **LOOPS**, a port of the Loops plugin (`../PLUGINSto112/Loops 2`).

LOOPS is a chance sampler. It keeps two seconds of sound and turns them into loops with three layers. Every **CHANCE** deals a new loop: each layer's region of the recording, its direction, pitches, place in the stereo field and echo send, all generated from a loop number.

Each layer is one of three types:

- **LOOP** repeats its region, with drift, gaps and a seam from raw clicks to soft crossfades.
- **STRETCH** plays the region slowly: aligned grains that keep the pitch, or a TAPE playhead whose pitch drops.
- **SPARKLE** picks short grains from the region, in sparse bursts.

Then each layer goes through its own speaker (direct, radios, telephone, megaphone, tin can...) and travels across the stereo field, into a ping-pong echo and a reverb wash.

It was built from NULLSTATION (`addac112_nullstation`), with the RADIO engine replaced by LOOPS. The looper, banks, SD card, presets, loop pitch, sync and REC prob/delay are the same as in the stock firmware.

```
engine/     loops_engine.* (DSP), analysis.* (recording analysis, pitch), speaker.h, dsp.h,
            loops_param_map.h (knobs, settings, preset keys); no firmware dependencies
reference/  the plugin's engine, presets and tests, unchanged (the A/B reference)
test/       desktop tests of the engine: ./run_tests.sh; Cortex-M7 cycle estimate: m7/
firmware/   the 112 firmware (src/, web/ editor, Makefile)
banks/      BANK0_LOOPS: the plugin's 16 factory presets as a 112 bank
tools/      make_factory_bank.sh: builds it from reference/Presets.h
```

```
cd firmware
make                # build/LOOPS_<version>.bin (git hash, or the build date)
make program-app    # bootloader in DFU mode
```

It builds against addac112's libdaisy / DaisySP / CMSIS (`make ADDAC112_DIR=...` to override).

Switching engines

Each engine is its own firmware. This one always runs LOOPS. Plain granular is the stock addac112 firmware.

1. Put the firmware `.bin` files (112, CD SKIP, FRIPPER, NULLSTATION, LOOPS, ...) in a folder named `ENGINES` on the SD card.
2. Go to **Menu** → **ENGINE** (the last item of the main menu). It lists the `.bin` files in `ENGINES`.
3. Choose one and confirm **FLASH & RESTART? YES**.

Presets store both engines' values: the stock grain keys plus `loops_*` keys. The stock 112 firmware ignores the `loops_*` keys, but if it saves a preset, those keys are dropped from that preset.

LOOPS controls (dual-label overlay)

[L]: the knob edits the layer the LAYER switch points at.

Row	Col 1	Switch	Col 2	Col 3
1	POSITION / **TYPE** [L]	START GRID / **LAYER** (1 / 2 / 3)	POS DEV / **POS DEV** [L]	GRAINS ACTIVE / **ODDS** (jack in trigger mode: **CHANCE**)
2	PLAY LENGTH / **LENGTH** [L]	SIZE GRID / **ECHO SYNC** (FREE / ÷8 / ÷4)	LENGTH DEV / **LEN DEV** [L]	ATTACK / **A** [L]

Row	Col 1	Switch	Col 2	Col 3
3	GRAIN DELAY / **PITCH**	SKIP CLOCK / **AUTO** (OFF / MID / RIGHT)	DELAY DEV / **ECHO**	DECAY / **B** [L]
4	DIRECTION / **REVERSE** [L]	REPEAT / **FATE** (RANDOM / FIXED)	PROB/REPEAT / **C** [L]	GRAIN PITCH / **TRANSPOSE** (+1V/oct)
5	VOLUME MIN / **LEVEL** [L]	–	VOLUME DEV / **WASH**	PANNING / **SPREAD**

A, B and C depend on the layer's type:

	A (ATTACK)	B (DECAY)	C (PROB/REPEAT)
LOOP	DRIFT: the start creeps on each repeat	GAP: the chance a repeat is silent	FADE: the seam, CLICK (0) to 200 ms
STRETCH	SPEED: 1/16x to 1x (TAPE: the pitch, live)	GRAIN: 20 ms to 1 s	SMOOTH: grainy to seamless
SPARKLE	RATE: 0.5 to 40 grains/s	SIZE: 5 to 400 ms	BURST: the chance of quick clusters

Knobs

- **TYPE**: LOOP / STRETCH / SPARKLE (three steps). The layer's other values stay, so turning back restores the old type as it was.
- **LENGTH**: the layer's region, 5 ms to 2 s (300 ms at the centre). Each loop deals a variation around it. Under about 20 ms a LOOP layer becomes a pitched hum.
- **POS DEV / LEN DEV**: the region's start and length wander over time, up to a second and two octaves either way. WANDER (settings) sets how often they head somewhere new.
- **REVERSE**: the chance a repeat, grain or tape pass plays backwards.
- **LEVEL**: the layer's level.
- **PITCH**: how far the layers are transposed when a loop is dealt. 0 % keeps the recording's pitch; 100 % gives wide intervals that also change within a loop. STRETCH leans low, SPARKLE high.
- **ECHO**: the level of the ping-pong tape echo. Each layer's send is dealt with the loop.
- **WASH**: the reverb haze.
- **SPREAD**: how wide the layers travel.
- **ODDS**: the chance each AUTO step deals a new loop.
- **TRANSPOSE** is the stock GRAIN PITCH: –24 to +36 semitones, the GRAIN PITCH CV at 1V/oct, quantized to the QUANTIZE scale (QUANTIZER: BOTH or LAYERS ONLY). It moves every layer: LOOP and SPARKLE on their next repeat or grain, a TAPE layer right away (with a 40 ms glide, like a motor).
- LENGTH and a STRETCH layer's SPEED move in steps larger than the pot noise (0.3 % of the travel), so a TAPE layer's pitch does not wobble.

Switches

- **LAYER**: which layer the [L] knobs edit. When it changes, those knobs wait until they cross the new layer's values (`PICKUP` on the display). So do A, B and C when TYPE changes.
- **ECHO SYNC**: FREE uses ECHO TIME (settings); ÷8 / ÷4 set the echo time to that slice of the selected loop. It follows the loop pitch.
- **AUTO**: OFF, or a new loop every AUTO MID / AUTO RIGHT (settings: one beat to eight bars, with the selected loop as one bar), each time with the chance ODDS.
- **FATE**:
  - RANDOM: every new loop is new.
  - FIXED: each SEED always gives the same loop, and CHANCE steps to the next seed. With AUTO, the loop follows the position in the loop's bars, so the same seed always plays out the same way.

Other panel controls in LOOPS mode

- **GRAINS VOLUME** is the LOOPS level.

- **GRAINS FEEDBACK** records LOOPS into the loop.
- **GRAINS trigger out** sends a pulse on each new loop (TRIGGER OUT +EVENTS: also on every repeat, grain and tape pass).
- **Encoder + STOP** (special function, enabled in Options) is **RECORD**: it captures two seconds (CAPTURE: MANUAL), or arms / disarms ON LEVEL and NEXT BAR.
- **CHANCE**: the GRAINS ACTIVE jack with CHANCE INPUT on (the grain trigger mode). Each rising edge deals a new loop (FIXED: the next seed).

Time While a loop plays, its length is one bar of BEATS / LOOP beats (default 4), and its start is the downbeat, whatever the loop pitch or direction. This drives AUTO, NEXT BAR and ECHO SYNC. With no loop playing, the beat is the TEMPO setting.

Settings

- Two quick pages, scrolled from the main screen:
  - **LOOPS SETTINGS**: SOURCE, CAPTURE, FEEDBACK, CHANCE INPUT, TRAVEL, CLICKS, ROOT, SEED.
  - **LOOPS LAYER n**: the settings of the layer the LAYER switch points at: SPEAKER, STRETCH MODE, SPARKLE SPEED, STRETCH FREE, SPARKLE FREE, STRETCH DICE, SPARKLE DICE, MUTE.
- Every setting, by group, under **Menu** → **Options** → **5.LOOPS OPTIONS**:
  - **CAPTURE**: SOURCE (LIVE / LOOP as heard / BOTH), CAPTURE (MANUAL / ON LEVEL / NEXT BAR / ALWAYS), THRESHOLD, INPUT, MIC, FEEDBACK (overdub), NORMALISE.
  - **CHANCE**: CHANCE INPUT, SEED, AUTO MID, AUTO RIGHT, PITCH MODE (ROOT / RANDOM), ROOT.
  - **SPACE**: TRAVEL, WANDER, CLICKS, CLICKS POST.
  - **ECHO + TIME**: ECHO TIME, ECHO FEEDBACK, BEATS / LOOP, TEMPO, TRIGGER OUT.
  - **LAYER 1 / 2 / 3**: SPEAKER (13 models), STRETCH MODE (STRETCH / TAPE), SPARKLE SPEED, STRETCH FREE, SPARKLE FREE, STRETCH DICE, SPARKLE DICE, MUTE.
- **DICE**: every new loop (CHANCE or AUTO) also deals that layer a new SPEED, from the loop number, so FIXED fate repeats it. The knob then waits to pick it up.

Display (LOOPS)

- **Main screen:**
  - A header with the focused layer and its type, and the loop number. It shows `SEED n` in FIXED fate, `REC nn%` while recording and `ARMED` while waiting.
  - A lane per layer:
    - its region of the two seconds, thicker for the focused layer, dotted for the others;
    - its grains as dots above it;
    - a dot at the left edge on each event;
    - a dotted line when the layer is muted.
  - The waveform of the recording. While recording, the new one is drawn up to where it is, with the old one dotted after it.
- **Left bar:** the input level (–60 to 0 dB), with a line at THRESHOLD in ON LEVEL. While recording: the progress, blinking.
- **Right bar:** SPREAD.
- **Top bar:** the focused layer's type (`LP` / `ST` / `SP`), or `CH` with the CHANCE input (inverted while it is high).
- **Bottom bar:**
  - the FATE (`RND`, or `FIX` inverted);
  - a meter per layer (the focused one filled);
  - G = TRANSPOSE.
- **Readout:** turning an overlay knob or flipping a switch shows its name and value for 1.5 s, with `PICKUP` until the knob reaches the value.

The recording

- **SOURCE**: what a capture records: the LIVE input, the LOOP as you hear it, or BOTH. VOL DRY stays clean.

- The recording is two seconds, mono.
  - It is normalised (NORMALISE, up to +30 dB).
  - Its DC is removed and its ends fade over 4 ms.
  - Its onsets are found: SPARKLE grains start on them.
- **FEEDBACK** is overdub: the recording that was playing is added in at this level, scaled back if the sum would clip.
- **CAPTURE ALWAYS** keeps recording, replacing the sample every two seconds while the layers play on.
- The analysis of a finished recording is spread over the next blocks (about 0.25 s at 48 kHz), so it never costs one big block. The new recording comes in after that, crossfaded; TAPE passes carry on from the same place in it.
- **ROOT** retunes the recording to a key by its detected pitch. The pitch is only tracked when ROOT is set (about 0.85 s per recording at 48 kHz).

Sound against the plugin

The engine is the plugin's DSP, with:

- caller-owned memory (SDRAM) instead of the heap;
- no locks;
- fast sine, pow and log on the audio path;
- the 112 additions, each of which can be switched off (`Params::exactAU`).

`test/run_tests.sh`:

- **ab_test** renders 41 cases on both engines with the same input, seed and loop number: each type on each layer, the 16 factory presets, every capture mode, overdub, both fates, Auto, clicks, echo sync, mic.
  - With the plugin's arithmetic (`–DLOOPS_FAST_MATH=0`): identical, sample for sample.
  - With the fast math the 112 runs: –95 to –140 dB below the signal, the level the same to 0.1 dB.
    - The exception is *Hum the root*: the detected pitch moves by rounding, so the retuned phase drifts. The level and zero crossings are the same.
  - With the 112 additions on: the same level (±0.6 dB) and brightness.
- **The plugin's own tests** (`reference/engine_test.cpp`, unchanged, through `test/plugin_api/`) pass on the 112 engine, both as the plugin and with the 112 additions on. They cover:
  - pitch detection, types, mute and deviation;
  - fates and capture modes;
  - save / restore, Tape, travel and wash;
  - overdub, Sparkle speed, clicks and dips;
  - all 13 speakers.
- **features_test** covers the 112 additions:
  - the sliced analysis gives the same buffer and onsets;
  - the budgeted pitch tracker gives the same note;
  - the worst block when a recording ends goes from 492 to 38 µs on desktop;
  - pitch is tracked only with ROOT;
  - the grain cap holds without clicks;
  - TRANSPOSE works on every type;
  - half rate at 96 kHz;
  - buffer load.
- **rate_test** covers every sample rate of the 112 (8 to 96 kHz).
- **param_map_test** checks the knobs, settings and preset keys.
- The web editor tests check that the editor's `loops_*` keys and defaults are exactly the firmware's.

A bug in the plugin, fixed in the port and applied to the reference for the A/B (`test/plugin_wrap_fix.patch`):

- Where: the echo, the travel delay line and the speaker comb.

- What: `rp = w - t` can round to exactly the line's length once wrapped, and the plugin then reads one sample past the end.
- What you hear:
  - in the echo, a sample of leftover heap memory (the plugin's output changes from run to run);
  - in the travel line, a one-sample dropout.
- The fix is three lines; worth applying to the plugin too.

CPU

The engine files are built with `-O2` (the rest of the firmware, like the stock 112, with `-Og`).

These are estimates from running the Cortex-M7 build in an emulator (`test/m7/m7_cycles.py`, 128-sample blocks): executed instructions, with VDIV/VSQRT weighted at 14 cycles. They ignore SDRAM and cache stalls, so read them as ±30 %.

Case (48 kHz)	Average	Worst block
defaults (Loop / Stretch / Sparkle)	~29 %	~39 %
Dense	~29 %	~49 %
Glass sparkle	~20 %	~22 %
Slow pad (two Tape / Stretch layers)	~34 %	~45 %
a recording ends (the analysis after it)	~36 %	~52 %
ROOT set: pitch tracking after a recording	~52 %	~63 %
worst: 2 Sparkle storms + Stretch, 3 heavy speakers, mic, clicks post (33 grains)	~73 %	~84 %
the same, the CPU guard at 12 grains	~43 %	~54 %
the same at 96 kHz (half rate inside), 12 grains	~50 %	~72 %

- A grain costs about 100–150 cycles per sample. A layer through a heavy speaker (telephone, megaphone, broken cone) costs about 230.
- **CPU guard**:
  - Above 85 % CPU, the bridge lowers the number of grains that may play at once, from 48 down to 12.
  - Below 65 % it raises it again, slowly.
  - New grains are skipped; playing ones are never cut, so it never clicks.
  - Typical settings play fewer than 12 grains anyway.
- At **88.2 / 96 kHz** the engine runs at half the rate inside, with 47-tap half-band filters in and out (from NULLSTATION).
- Port optimisations, each without changing the plugin-exact output (or only under the fast math):
  - the Wander curve once per block, not three `pow` per sample;
  - the Stretch speed once per block;
  - reciprocal glides and envelopes;
  - the region walkers stepped every 16 samples (112 mode only);
  - the speaker drive without `sqrt` and division, and the telephone's mu-law with fast log / exp.

Memory

	GRANULAR (stock)	LOOPS
Loop memory	360 s	~364 s (mono at 44.1 kHz)
Engine buffers (2.6 MB: three 2 s recordings, echo, wash, analysis, display copy)	–	top of the loop array in SDRAM

	GRANULAR (stock)	LOOPS
Grains, layers, speakers + bridge (~60 KB)	–	heap (RAM_D2), created at boot
Grain voice reserve (21 KB heap)	yes	no

- DTCM is at 89.3 % (stock: 88.6 %).
- The binary is 433 KB of the 480 KB the bootloader copies to SRAM.

Factory bank

`banks/BANK0_LOOPS` holds the plugin's 16 factory presets as 112 presets (`1.CFG` to `16.CFG`).

- To use it, copy the folder to the SD card. The 112 reads it as bank 0 (only the number after `BANK` counts); rename it if bank 0 is taken.
- It has no loops: record a sample (Encoder + STOP), or add WAV files to its `WAV` folder.
- `NAMES.txt` lists the presets by number. On the 112, FATE, AUTO and ECHO SYNC are switches, so it also says which switch positions a preset was made for:
  - *Fixed fate, bar by bar*: FATE on FIXED, AUTO on MID.
  - *Endless replace* and *Dice speeds*: AUTO on MID.
  - *Glass sparkle*: ECHO TIME holds its dotted eighth at 120 BPM.
- Each preset sets every `loops_*` value. The rest (volumes, loop pitch, options) comes from the bank's defaults.
- `tools/make_factory_bank.sh` rebuilds it from `reference/Presets.h` with the firmware's own preset writer, and checks that every file reads back the same.

Web editor

`firmware/web/` has four Loops sections next to Grains: Loops (the global values) and Loops layer 1–3. Each value has its help text.

Run the editor's tests with `for t in firmware/web/test/*.test.js; do node "$t"; done`. The sample files are read from `../addac112/support`. The tests check two things:

- the editor's `loops_*` keys are exactly the ones `loops::saveKeys()` writes;
- its defaults are the engine's (`test/run_tests.sh`).

Not done yet

- Not yet run on hardware. Things to check:
  - CPU % in the worst cases above, and the guard;
  - heap headroom with large banks (the bridge is ~22 KB more than NULLSTATION's);
  - the pot deadband on LENGTH and SPEED;
  - LAYER pickup feel;
  - the CHANCE input, Encoder + STOP RECORD, the trigger out;
  - AUTO / NEXT BAR / ECHO SYNC timing against the loop;
  - the display at 50 fps.
- Overlay artwork: needs the panel dimensions and knob positions.